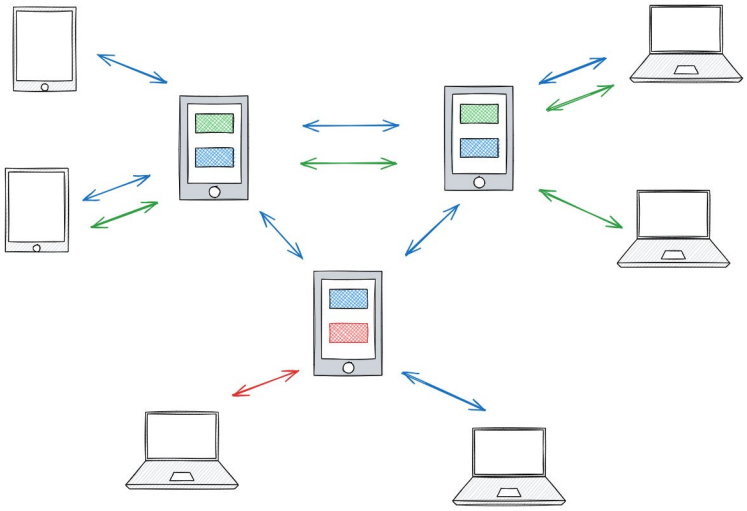
Exercise 1 - Distributed Systems

A Naive View

Spencer Johnson (spencerjames.johnson)



Abstract

This report describes a socket-based distributed counter system. The objective is to share an integer variable between several server processes without using semaphores, shared memory, message queues, or other IPC mechanisms. The implemented solution uses socket frames and a token ring to control when each server can read or update the shared value.

laSalle
RAMON LLULL UNIVERSITY

Distributed Systems

La Salle URL - Barcelona, Spain

2nd October 2026

Contents

1 Introduction **2**

2 System Architecture **2**
2.1 Processes Overview 2
2.2 Server Functionality 2
2.3 Ring Structure 2

3 Communication Protocol **3**

4 Algorithm **3**
4.1 Algorithm 1: Neighbor Read 3
4.2 Algorithm 2: Broadcast Update 4

5 Results **5**
5.1 Testing Configurations 5
5.2 Performance Overhead 5
5.3 Large Ring Test 6

6 Limitations **6**

7 Adding Servers Mid-Run **6**

8 Conclusion **6**

1 Introduction

The goal of this exercise is to share one integer variable between `N` server processes. The servers are only allowed to communicate through sockets. Each server has one behavior selected at startup: read-write or read-only.

The integer value starts at `0`. Read-write servers read the current value and update it with `value + 1`. Read-only servers only read the current value. Each server performs this behavior for a fixed number of turns.

2 System Architecture

2.1 Processes Overview

- **menu.js**: Starts all servers and builds the ring configuration.
- **server.js**: Represents one server in the distributed system.

2.2 Server Functionality

- **Read-write server**: Reads the value and updates it by one.
- **Read-only server**: Reads the value without changing it.

2.3 Ring Structure

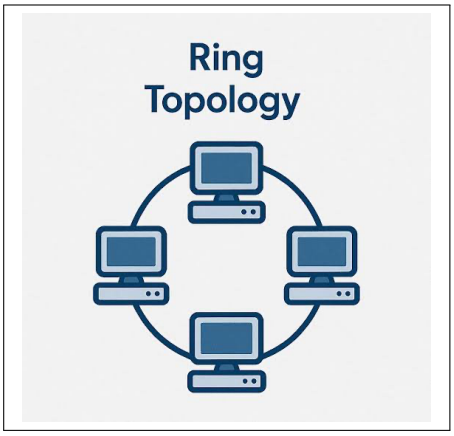


Figure 1: Ring topology used for token passing between servers.

Each server knows the previous server and the next server. The token moves from one server to the next. A server can only perform its turn after receiving the token.

3 Communication Protocol

The system communicates using simple socket frames.

- `READ`: Returns the current local value.
- `UPDATE <value>`: Updates the current local value.
- `TOKEN`: Gives a server permission to take its turn.

`READ` and `UPDATE <value>` are the two operations on the shared variable. The `TOKEN` frame is used only to control turn order.

4 Algorithm

4.1 Algorithm 1: Neighbor Read

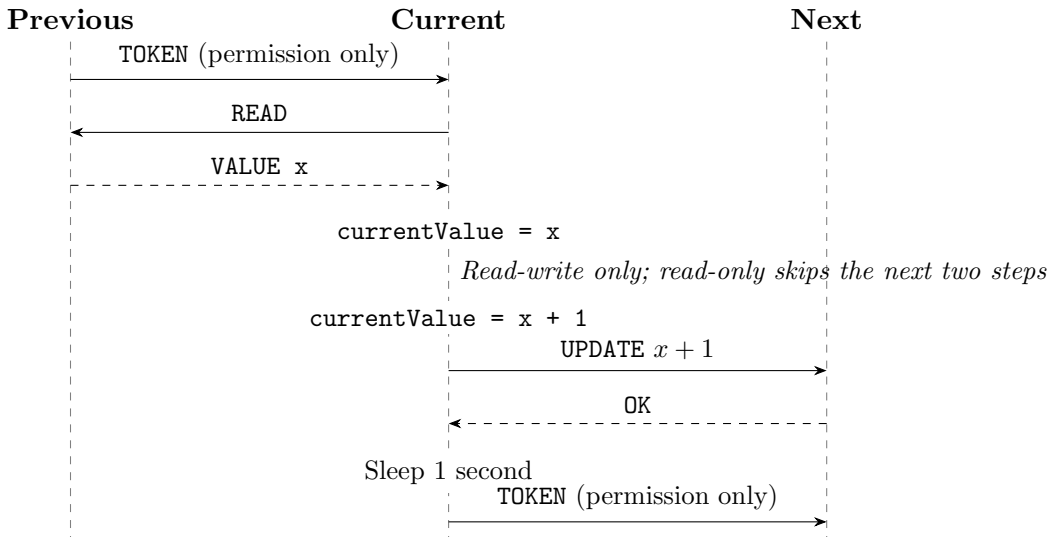


Figure 2: One active turn in Algorithm 1. `UPDATE` carries the computed integer $x + 1$. Token acknowledgements are omitted for clarity.

4.2 Algorithm 2: Broadcast Update

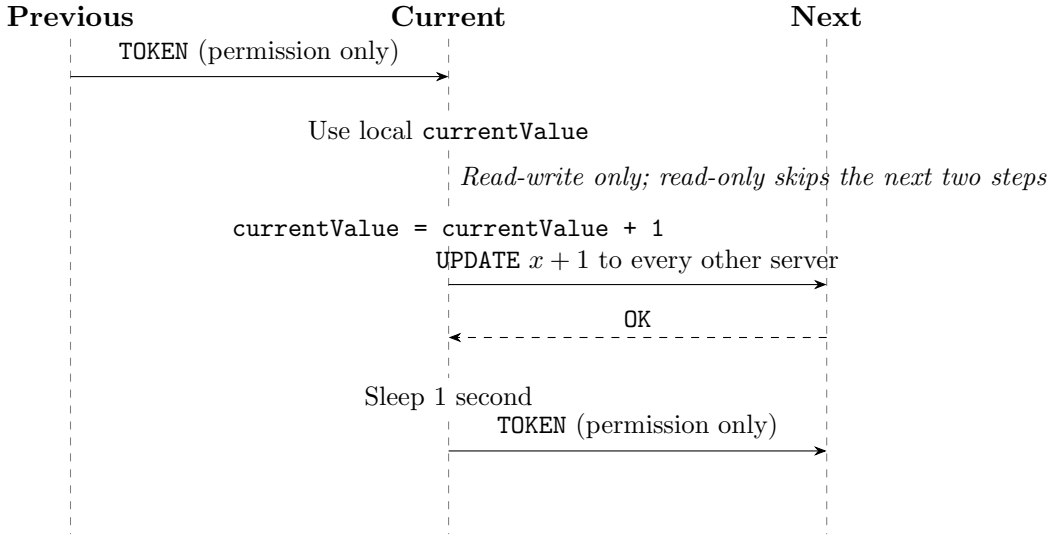


Figure 3: One active turn in Algorithm 2. A read-write server broadcasts its new local value before passing the token.

5 Results

5.1 Testing Configurations

- 100 servers.
- 75 read-write servers and 25 read-only servers.
- 10 turns per server, with a 1000ms sleep during every turn.
- Algorithm 1 and Algorithm 2 were run separately using the same configuration.

5.2 Performance Overhead

The timing results were collected from the terminal output. Each completed turn records the UTC timestamp, server port, server type, turn number, read value, and local value:

```
2026-10-13T23:55:55.012Z | Server 5002 |
ReadWrite | Turn 1 | Read 0 | Value 1
```

The execution time was measured from the first turn of server `5002` until server `5001` completed its tenth turn. Every 100-server execution contains 1000 required one-second sleeps, so the fixed 1000 seconds were subtracted from the measurements below. Both tests finished with the expected value of `750`, since 75 read-write servers each incremented the value 10 times.

Algorithm	Servers	Final value	Time excluding sleep (s)
Algorithm 1: Neighbor Read	100	750	6.991
Algorithm 2: Broadcast Update	100	750	42.926

Table 1: Communication and processing time after removing the required sleep time.

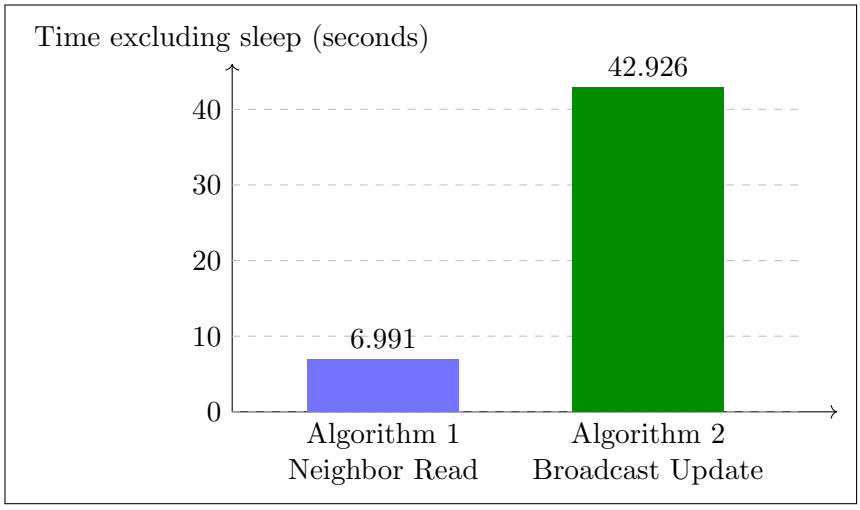


Figure 4: Communication and processing time for the completed 100-server tests, excluding the required sleep time.

After removing the required sleep time, Algorithm 2 took 35.935 seconds longer than Algorithm 1. The extra time comes from each read-write server sending an `UPDATE` frame to every other server.

5.3 Large Ring Test

The 750-server tests completed one full rotation before being stopped. To compare the ring sizes, the required sleep time was removed from the duration of that first rotation. The 100-server test used 75 read-write servers, while the 750-server test used 20 read-write servers.

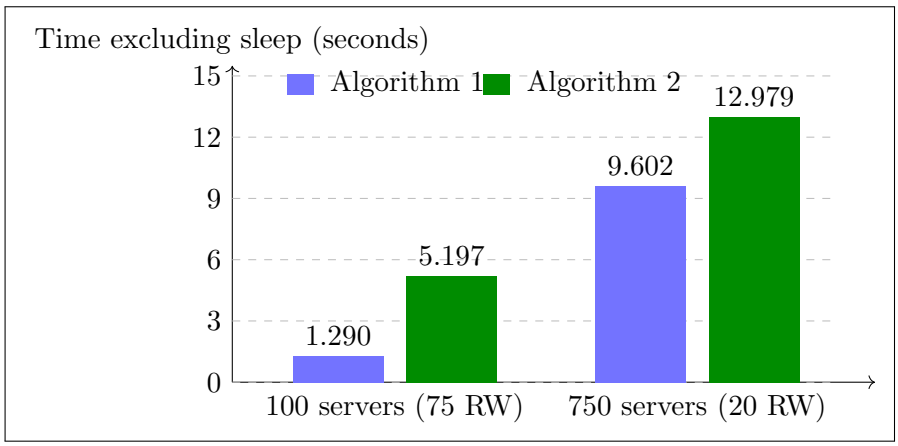


Figure 5: Communication and processing overhead during one full ring rotation. The 750-server result covers the completed first rotation only.

6 Limitations

- The token ring depends on all servers staying alive.
- If one server fails, the token may stop moving.
- Adding servers during execution is not currently supported.
- Larger rings increase the time needed for the token to return to the same server.

7 Adding Servers Mid-Run

To add servers during execution, the ring would need a way to safely update neighbor information. A joining server would need to be inserted between two existing servers without losing the token or breaking the current socket communication.

This would require extra control frames for joining, leaving, and updating neighbor addresses. It would also make the algorithm more complex because every server would need to agree on the new ring structure.

8 Conclusion

The implemented system follows the assignment restrictions by using sockets for communication and avoiding shared memory, semaphores, message queues, and other IPC mechanisms. The token ring provides a simple way to control access to the shared integer, while the two algorithm versions allow performance comparison between neighbor-based reading and broadcast-based updating.